

The existence of a file can be tested with the `doesFileExist` function. You can check if a directory is present using the `doesDirectoryExist` function. Their type signatures are:

```
ghci> :t doesFileExist
doesFileExist :: FilePath -> IO Bool

ghci> :t doesDirectoryExist
doesDirectoryExist :: FilePath -> IO Bool
```

Some examples that use these functions are shown below:

```
ghci> doesDirectoryExist "/abcd"
False

ghci> doesDirectoryExist "/tmp"
True

ghci> doesFileExist "/etc/resolv.conf"
True

ghci> doesFileExist "/etc/unresolv.conf"
False
```

To know the current directory from where you are running the command, you can use the `getCurrentDirectory` function, and to know the contents in a directory you can use the `getDirectoryContents` function. Their type signatures are:

```
ghci> :t getCurrentDirectory
getCurrentDirectory :: IO FilePath

ghci> :t getDirectoryContents
getDirectoryContents :: FilePath -> IO [FilePath]
```

For example:

```
ghci> getCurrentDirectory
"/tmp"

ghci> getDirectoryContents "/etc/init.d"
["livesys", "netconsole", ".", "..", "network", "README",
"functions", "livesys-late", "influxdb"]
```

The `copyFile`, `renameFile` and `removeFile` functions are used to copy, rename and delete files. Their type signatures are shown below:

```
ghci> :t copyFile
copyFile :: FilePath -> FilePath -> IO ()

ghci> :t renameFile
```

```
renameFile :: FilePath -> FilePath -> IO ()
```

```
ghci> :t removeFile
removeFile :: FilePath -> IO ()
```

Here is a very contrived example:

```
import System.Directory
main = do
  copyFile "/etc/resolv.conf" "/tmp/resolv.conf"
  renameFile "/tmp/resolv.conf" "/tmp/resolv.conf.orig"
  removeFile "/tmp/resolv.conf.orig"
```

To obtain the file permissions, use the `getPermissions` function:

```
ghci> :t getPermissions
getPermissions :: FilePath -> IO Permissions

ghci> getPermissions "/etc/resolv.conf"
Permissions {readable = True, writable = False,
executable = False, searchable = False}
```

It is important to separate pure and impure functions in your code and to include the type signatures for readability. An example is shown below:

```
-- Pure
square :: Int -> Int
square x = x * x

-- Impure
main = do
  putStrLn "Enter number to be squared:"
  number <- readLn
  print (square number)
```

The `readLn` function is a parameterised IO action whose type signature is:

```
:t readLn
readLn :: Read a => IO a
```

Executing the code produces the following output:

```
ghci> main
Enter number to be squared :
5
25
```



By: Shakthi Kannan

The author is a free software enthusiast and blogs at shakthimaan.com.